

```
print('Original dataset shape %s' % Counter(y))
Original dataset shape Counter({1: 894, 0: 106})

# scatter plot of examples by class label
for label, _ in Counter(y).items():
    row_ix = where(y == label)[0]
    pyplot.scatter(X[row_ix, 0], X[row_ix, 1], label=str(label))
pyplot.legend()
pyplot.show()
```

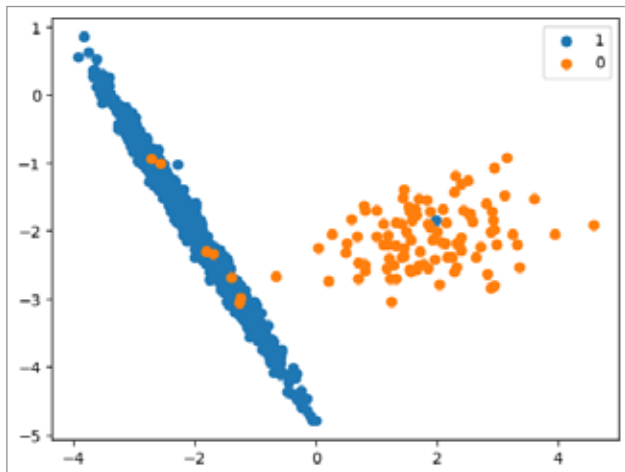


Figure 1: Scatter plot of examples by class label – pre-downsampling (used the same data for all the downsampling methods, and observe how this data changes after using each technique)

```
# perform downsampling
df = pd.DataFrame(X, columns=['Column_A', 'Column_B',
                             'Column_C', 'Column_D'])
df['Y'] = y
over_samples = df[df["Y"] == 1]
under_samples = df[df["Y"] == 0]
print("Original Over Sample Shape", under_samples.shape)
df_res = resample(over_samples,
                  replace=True,
                  n_samples=len(under_samples),
                  random_state=42)
print("Resampled Over Sample Shape", df_res.shape)

X_Res = np.append(df_res[['Column_A', 'Column_B', 'Column_C',
                           'Column_D']].to_numpy(), under_samples[['Column_A',
                           'Column_B', 'Column_C', 'Column_D']].to_numpy(), axis=0)

y_res = np.append(df_res['Y'].to_numpy(), under_samples['Y'].to_numpy())

print('Resampled dataset shape %s' % Counter(y_res))
Original Over Sample Shape (106, 5)
Resampled Over Sample Shape (894, 5)
```

```
Resampled dataset shape Counter({0: 894, 1: 894})

# scatter plot of examples by class label
for label, _ in Counter(y_res).items():
    row_ix = where(y_res == label)[0]
    pyplot.scatter(X_Res[row_ix, 0], X_Res[row_ix, 1],
                  label=str(label))
pyplot.legend()
pyplot.show()
```

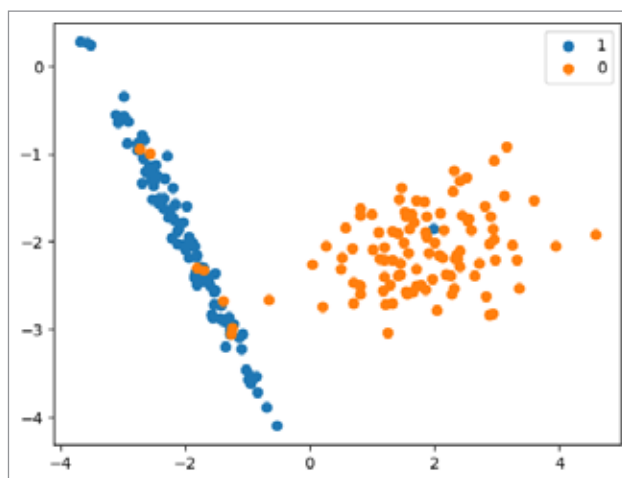


Figure 2: Scatter plot of examples by class label downsampled post using resample

RandomUnderSampler

- In this method, the rows from the majority class are selected randomly and replaced.
- Logic used is the same as that used for upsampling, except the strategies in this case impact the majority class instead of the minority class.
- The source code is:

```
from collections import Counter
from sklearn.datasets import make_classification
from imblearn.under_sampling import RandomUnderSampler
from matplotlib import pyplot
from numpy import where

X, y = make_classification(n_classes=2, class_sep=2,
                          weights=[0.1, 0.9], n_features=4, n_clusters_per_class=1,
                          n_samples=1000, random_state=10)

# summarize class distribution
print('Original dataset shape %s' % Counter(y))
Original dataset shape Counter({1: 894, 0: 106})
```

Now we will carry out downsampling: